

# Lesson 3: Event Listeners — Responding to User Actions

## Mission Brief

A ship with buttons nobody can press is just decoration. Today the Lumexa Explorer's control panel comes alive: clicking a button fires the thrusters, typing in a field updates a readout in real time, and pressing Enter submits a command. You're about to learn **events** — the mechanism that lets your JavaScript react the instant a human does something.

## Learning Objectives

By the end of this lesson, you will be able to:

- Explain what a DOM event is and name common event types (`click`, `input`, `submit`, `keydown`)
- Attach event listeners with `addEventListener()` and remove them with `removeEventListener()`
- Use the `event` object to read information about what happened (e.g., which key was pressed, the input's current value)
- Prevent a form's default page-reload behavior with `event.preventDefault()`
- Use **event delegation** to handle events for elements that don't exist yet

## What You'll Build

An interactive "Command Console": a text input and button where typing updates a live preview as you type, clicking "Send Command" logs the command to an on-page history list, and pressing Enter in the input does the same thing as clicking the button — all wired up with event listeners.

## Prerequisites

- Lesson 1 (variables, functions)
- Lesson 2 (selecting and creating DOM elements)

## Key Concepts

- Events and the event loop (conceptually)
- `addEventListener(type, handler)`
- The `event` object (`event.target`, `event.key`, `event.preventDefault()`)
- Common event types: `click`, `input`, `change`, `submit`, `keydown`
- Event delegation

- Removing listeners with `removeEventListener`

## Concept Explanation

An **event** is something that happens in the browser that JavaScript can be notified about — a user clicking a button, typing a character, submitting a form, resizing the window, or a page finishing loading. JavaScript lets you **listen** for these events and run a function (called a **handler** or **callback**) whenever they occur, using `element.addEventListener("eventType", handlerFunction)`.

Every time an event fires, the browser passes your handler function an **event object** containing details about what happened. For a `click` event, `event.target` tells you exactly which element was clicked (useful when multiple elements share one handler). For a `keydown` event, `event.key` tells you which key was pressed (e.g., `"Enter"`, `"a"`, `"ArrowUp"`). For an `input` event on a text field, you can read the current text with `event.target.value`.

Forms have a special behavior: submitting one (by pressing Enter in a field, or clicking a submit button) triggers a `submit` event and then **reloads the page** by default — almost never what you want in a modern interactive app. Calling `event.preventDefault()` as the very first line of your submit handler stops that reload so you can handle the data yourself with JavaScript.

A powerful and slightly advanced pattern is **event delegation**: instead of attaching a listener to every individual item in a list (especially items that don't exist yet, like to-do items added later), you attach *one* listener to a stable parent container, and inside the handler you check `event.target` to figure out which child was actually interacted with. This solves a real problem: `addEventListener` only works on elements that exist at the time you call it, so newly created elements (like a new to-do item) need either their own listener attached at creation time, or delegation from a parent that was already there.

Finally, you can undo a listener with `element.removeEventListener("eventType", handlerFunction)` — but note it only works if you pass the *exact same function reference* you used in `addEventListener`, which is why handlers are often defined as named functions rather than anonymous ones when you plan to remove them later.

## Guided Coding

```

<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Command Console</title>
  <style>
    body { font-family: sans-serif; background: #0b0e1a; color: #e6ecff; padding: 2rem; }
    #preview { color: #93c5fd; min-height: 1.5em; }
    #history li { padding: 0.2rem 0; }
    button { cursor: pointer; }
  </style>
</head>
<body>
  <h1>Command Console</h1>
  <input type="text" id="command-input" placeholder="Type a command..." />
  <button id="send-btn">Send Command</button>
  <p>Live preview: <span id="preview"></span></p>
  <h2>Command History</h2>
  <ul id="history"></ul>
  <script src="script.js"></script>
</body>
</html>

```

```

// script.js

const commandInput = document.querySelector("#command-input");
const sendButton = document.querySelector("#send-btn");
const preview = document.querySelector("#preview");
const historyList = document.querySelector("#history");

// --- LIVE PREVIEW WITH THE "input" EVENT ---
// Fires every time the text field's value changes (typing, pasting, deleting)
commandInput.addEventListener("input", (event) => {
  preview.textContent = event.target.value;
});

// --- SHARED LOGIC FOR SUBMITTING A COMMAND ---
function submitCommand() {
  const commandText = commandInput.value.trim();

  if (commandText === "") {
    console.log("Ignoring empty command.");
    return; // guard clause: don't add blank history entries
  }

  const listItem = document.createElement("li");
  listItem.textContent = commandText;
  historyList.appendChild(listItem);

  // Reset the input and preview for the next command
  commandInput.value = "";
  preview.textContent = "";
}

// --- CLICK EVENT ---
sendButton.addEventListener("click", () => {
  submitCommand();
});

// --- KEYDOWN EVENT: Enter key acts like clicking the button ---
commandInput.addEventListener("keydown", (event) => {
  if (event.key === "Enter") {
    event.preventDefault(); // stops any default form behavior
    submitCommand();
  }
});

// --- EVENT DELEGATION ---
// Clicking any history item (even ones added after this code ran) removes it.
// One listener on the stable parent handles all current AND future children.
historyList.addEventListener("click", (event) => {

```

```

if (event.target.tagName === "LI") {
  event.target.remove();
  console.log("Removed a command from history.");
}
});

```

## Code Walkthrough

- **Live preview:** the `input` event fires on every keystroke (unlike `change`, which only fires when the field loses focus). `event.target.value` reads the current text directly from the field that fired the event.
- **submitCommand():** pulled out as its own named function so both the click handler and the keydown handler can call the exact same logic — avoiding duplicated code. `.trim()` removes accidental leading/trailing spaces, and the guard clause skips empty submissions.
- **Click listener:** a simple arrow function that calls `submitCommand()` — deliberately kept as a wrapper so `submitCommand` doesn't need to know or care about the event object.
- **Keydown listener:** checks `event.key === "Enter"` to detect the Enter key specifically (not any other key), and calls `event.preventDefault()` defensively in case this input is ever inside a real `<form>` later.
- **Event delegation on `historyList`:** instead of attaching a "click to remove" listener to every `<li>` individually (which would miss any `<li>` added after page load), one listener sits on the parent `<ul>`. Inside it, `event.target` is whichever specific element was actually clicked, and the code checks `event.target.tagName === "LI"` before removing it.

## Student Mission

1. Add a "Clear History" button that removes all `<li>` elements from `#history` when clicked (hint: `historyList.innerHTML = ""` or loop and `.remove()` each child).
2. Add a character counter ( `<span id="char-count">0</span>` ) that updates live as the user types in `#command-input`, using the `input` event.
3. Disable the "Send Command" button ( `button.disabled = true` ) whenever the input is empty, and re-enable it once text is typed, using the `input` event to check the current value each time.

## Challenge

Add keyboard shortcuts: pressing `ArrowUp` while focused on `#command-input` should re-fill the input with the *last* submitted command (you'll need a variable to remember it). This mimics real terminal "command history" behavior and requires combining `event.key` checks with state you track between events.

## Experiment/Extension

In DevTools' Console, run `document.querySelector("#send-btn").addEventListener("click", () => console.log("second listener!"))` after the page has loaded, then click the button — notice that **both** your original listener and this new one fire. This proves an element can have multiple independent listeners for the same event type.

## Common Mistakes

- Forgetting `event.preventDefault()` inside a real `<form>`'s submit handler, causing an unwanted page reload that wipes out all JavaScript state.
- Confusing `input` (fires on every keystroke) with `change` (fires only when focus leaves the field) and being surprised the live preview doesn't update as expected.
- Attaching a listener directly to a to-do item's delete button at creation time, but forgetting that a *newly created* item after the page loaded needs its own listener too — a common bug that event delegation solves.
- Comparing `event.key === "enter"` (lowercase) instead of `"Enter"` (capital E) — JavaScript string comparisons are case-sensitive.
- Passing an anonymous arrow function to `addEventListener` and then being unable to `removeEventListener` it later, since a new arrow function is a different reference each time.

## Debugging Tips

- Add `console.log(event)` as the first line inside any handler to inspect the entire event object and discover properties you didn't know existed.
- In the Console, type `$0.addEventListener` (after selecting an element in the Elements tab, `$0` refers to it) to experiment interactively.
- If clicking a button seems to do nothing, check the Console for errors first — a thrown error earlier in the handler silently stops the rest of that handler from running.
- Use `debugger;` as a line inside your handler function, then trigger the event with DevTools open — execution pauses there, and you can inspect variables and step through line by line.

## Check Your Understanding

1. What is the difference between the `input` and `change` events on a text field?
2. Why do we call `event.preventDefault()` in a submit handler?
3. What problem does event delegation solve?
4. How does `event.target` differ from the element you originally attached the listener to?
5. Why might you extract shared logic (like `submitCommand()`) into its own named function instead of writing it twice?

## Mini Quiz + Answer Key

- Q1.** Which event fires on every keystroke in a text input? A) `change` B) `submit` C) `input` D) `keyup` only
- Q2.** What does `event.target` refer to inside a delegated click handler on a parent `<ul>`? A) Always the `<ul>` itself B) The specific child element that was actually clicked C) The whole document D) `undefined`
- Q3.** What does `event.preventDefault()` stop, when called in a form's submit handler? A) All future events B) The default page reload/navigation C) The event from reaching `event.target` D) Nothing, it's decorative
- Q4.** To detect the Enter key inside a `keydown` handler, you should check: A) `event.key === "Enter"` B) `event.target === "Enter"` C) `event.type === "Enter"` D) `event === "Enter"`

**Answer Key:** 1-C, 2-B, 3-B, 4-A

## Lesson Recap

You learned how to make a page respond to real user actions using `addEventListener`, how to read details from the `event` object, how to stop a form's default reload with `preventDefault()`, and how event delegation lets one listener handle clicks on elements that don't even exist yet.

## Homework

Build a small "quiz button" page: three buttons labeled A, B, and C, and a result area. Clicking each button should log which one was clicked and change the result text to say "You chose A" (or B/C) using a single delegated click listener on a parent container, rather than three separate listeners.

## Portfolio Project Connection

Every piece of interactivity in this course's projects depends on event listeners: adding/completing/deleting to-dos on click (Project 04), submitting a city name to search the weather API (Project 05), and detecting card clicks/flips in the memory game (Project 06) are all built directly on the patterns from this lesson.